

DOSSIER DE RENDU — INFRASTRUCTURE

Projet : Lency — SaaS, plateforme communautaire dans l'audiovisuel Équipe : Timothée Van Den Bosch - Guerric Cochelin

Domaine	URL
Production	https://lency.net
Staging	https://staging.lency.net

 **lency**
lency.net

chore(refactoring): Archi with Imports ...
Feb 17 on  main

 tim-vdb/Lency  ...

 **lency-staging**
staging.lency.net

Merge branch 'dev' into staging
20h ago on  staging

 tim-vdb/Lency  ...

1. Application déployée et fonctionnelle

Accessibilité

L'application est accessible en production sur **https://lency.net** et disponible en pré-production sur **https://staging.lency.net**.

Le déploiement est effectué via **Vercel** (PaaS), connecté au dépôt GitHub **tim-vdb/Lency** :

- Branche **main** → déployée automatiquement sur **lency.net** (production)
- Branche **staging** → déployée automatiquement sur **staging.lency.net** (staging)

▲ lency-staging ↕

Deployments

...

Status 5/6

8e5HgGSMS

Production 

● Ready

1m 23s

 staging

 a5e37ad Merge branch 'dev' into staging

20h ago by tim-vdb  ...

6fJtaj39D

Production 

● Ready

1m 21s

 staging

 790b386 hotfix(update parameter naming in userCon...

Apr 2 by tim-vdb  ...

DLTfS7j6h

Production

● Error

1m 13s

 staging

 ffe76e9 hotfix(parameter-naming-newsletter)

Apr 2 by tim-vdb  ...

2EDoR5oBs

Production

● Error

1m 17s

 staging

 528aca2 Merge branch 'dev' into staging

Apr 2 by tim-vdb  ...

FgGaAwgDC

Production 

● Ready

1m 18s

 staging

 47133b6 hotfix(suspense-useSearchParams)

Mar 18 by tim-vdb  ...

38ZXarvtp

Production

● Error

1m 7s

 staging

 4fcf183 Merge branch 'dev' into staging

Mar 18 by tim-vdb  ...

C6iG1459z

Production 

● Ready

1m 27s

 staging

 fa91366 Merge branch 'dev' into staging

Mar 11 by tim-vdb  ...

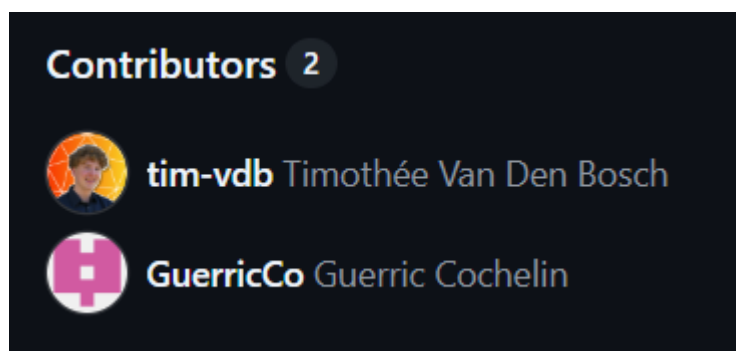
Base de données

La base de données est un **PostgreSQL serverless** hébergé sur **Neon.com**, connectée via **Prisma ORM**. Elle est opérationnelle sur les deux environnements avec des instances séparées (production / staging).

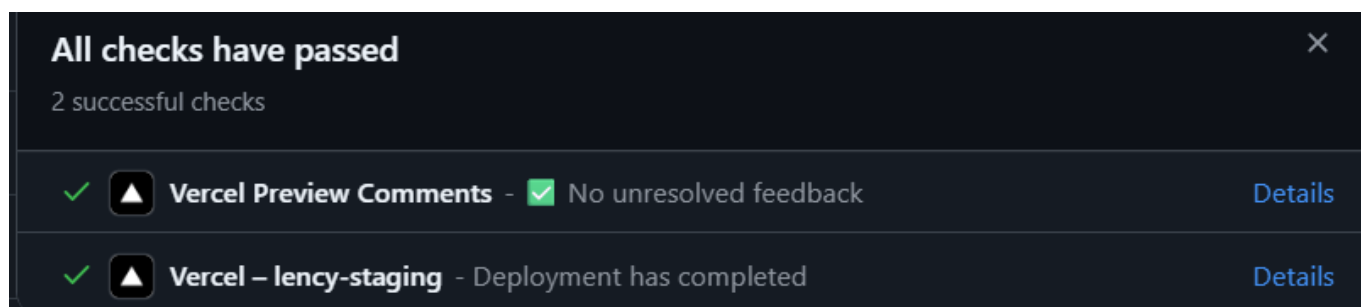
Name	Region	Created at	Storage	Compute last active at	Branches	Integrations
❖ CRA-MDS	AWS Europe Central 1 (Frankfurt)	Mar 23, 2026 12:09 pm	32.07 MB	Apr 14, 2026 11:35 am	0	Add ⊕ ⋮
❖ Lency	AWS Europe West 2 (London)	Mar 10, 2026 11:15 pm	30.82 MB	Mar 10, 2026 11:15 pm	1	Add ⊕ ⋮
❖ Lency-staging	AWS Europe West 2 (London)	Mar 10, 2026 11:10 pm	32.51 MB	Apr 17, 2026 8:07 am	1	Add ⊕ ⋮
❖ crudNextAPI	AWS US East 1 (N. Virginia)	Jan 31, 2026 7:31 pm	31.25 MB	Feb 11, 2026 4:05 pm	1	Add ⊕ ⋮
❖ Lency-dev	AWS US East 1 (N. Virginia)	Jan 20, 2026 8:26 am	41.96 MB	Apr 16, 2026 8:21 pm	1	Add ⊕ ⋮

Projet récupéré depuis GitHub

Le projet est lié au dépôt privé github.com/tim-vdb/Lency.



Chaque push sur une branche surveillée déclenche automatiquement un nouveau build et déploiement sur Vercel.



Front correctement servi

Le frontend est structuré avec **Next.js 15 (App Router)** compilée via **Turbopack**. Vercel sert les assets statiques depuis son CDN et les 33 routes dynamiques sont exposées via des **fonctions serverless Node.js 24.x**

2. Qualité de l'hébergement / architecture

Choix d'hébergement : PaaS — Vercel

Vercel a été retenu comme hébergeur **PaaS** pour sa compatibilité native avec Next.js, sa gestion automatique du scaling, du CDN et du SSL, et sa simplicité d'intégration CI/CD avec GitHub.

Architecture multi-environnements

```
GitHub (tim-vdb/Lency)
├─ branch main      → Vercel → lency.net      (Production)
├─ branch staging   → Vercel → staging.lency.net (Staging)
```

Chaque environnement dispose de :

- Son propre projet Vercel (**lency** / **lency-staging**)
- Sa propre base de données Neon.com
- Ses propres variables d'environnement injectées via **Doppler**

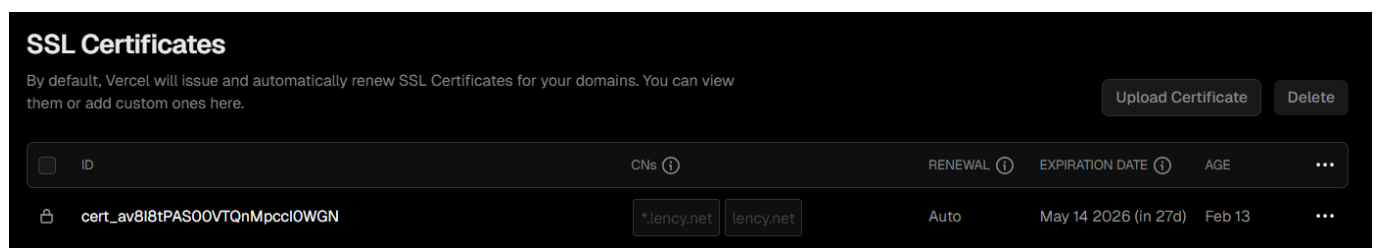
Architecture claire

- **Frontend** : Next.js 15, App Router, Turbopack
- **Backend** : API Routes Next.js → Fonctions serverless Node.js 24.x
- **Base de données** : Neon.com (PostgreSQL serverless) + Prisma ORM
- **Gestion des secrets** : Doppler (projets isolés prod / staging / development)
- **Emails** : Resend (transactionnel via **support@mail.lency.net**) + Proton Mail SMTP (**social@lency.net**)
- **Sous-domaines** :
 - **lency.net** — production
 - **staging.lency.net** — pré-production
 - **mail.lency.net** — envoi d'emails (SPF / DKIM / DMARC configurés dans Vercel / Resend)

3. Sécurité et bonnes pratiques

HTTPS

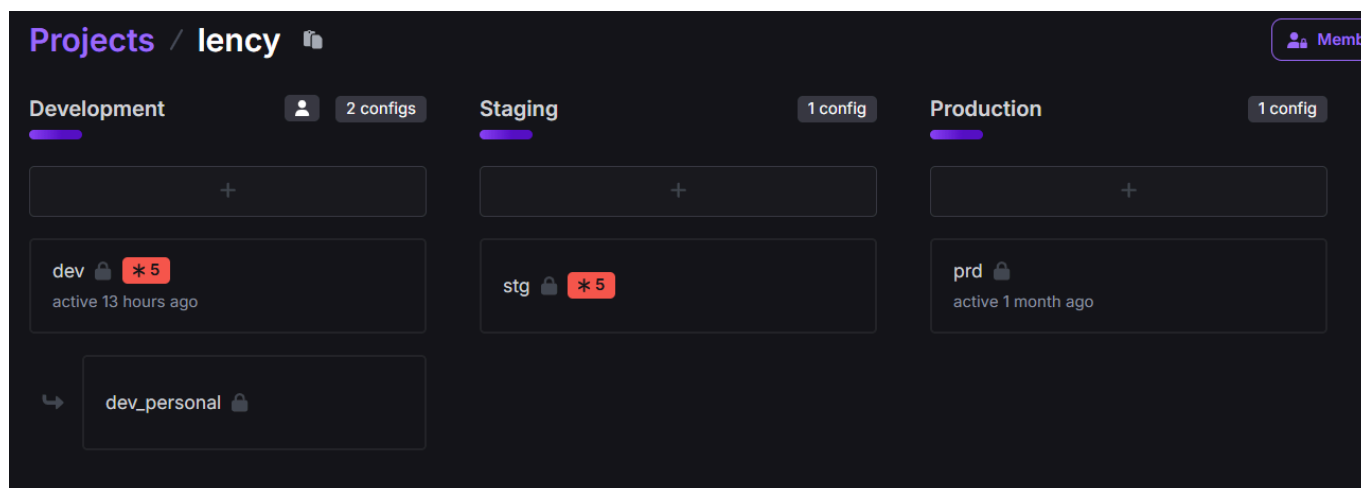
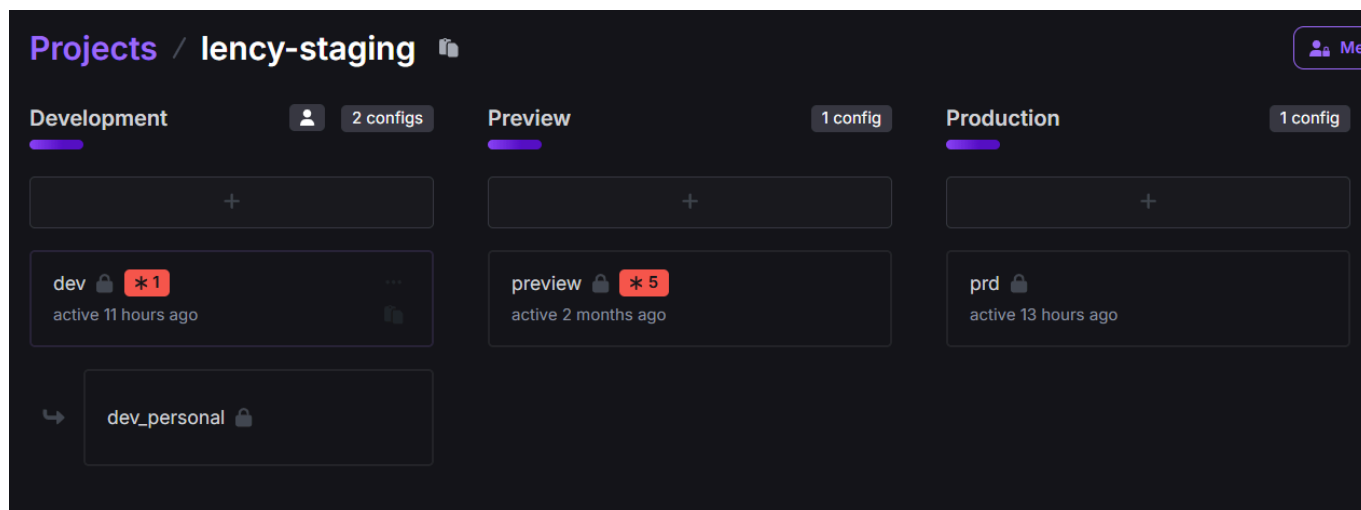
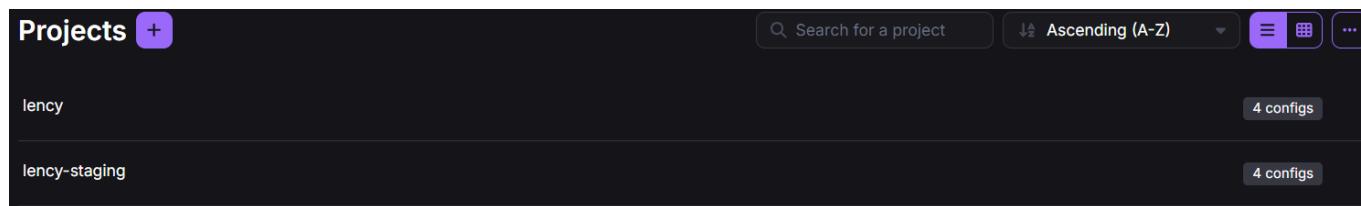
HTTPS activé automatiquement sur tous les domaines via les certificats SSL gérés par Vercel (Let's Encrypt). Aucune connexion HTTP non chiffrée n'est possible.



Secrets non exposés

Les variables d'environnement (clés API, chaînes de connexion DB, secrets d'auth) sont **exclusivement gérées via Doppler**. Elles ne sont jamais commitées dans le dépôt GitHub (**.env.local** présent dans **.gitignore**).

Doppler injecte les secrets au moment du build Vercel selon l'environnement (production ou staging et en local development lors du développement).



Debug désactivé en production

Le mode debug Next.js est désactivé en production. Les logs d'erreur sont disponibles uniquement via le dashboard Vercel (runtime logs) et non exposés publiquement.

Accès sécurisé

L'authentification est gérée via **Better Auth** (`/api/auth/[...all]`), avec sessions sécurisées. L'accès au dashboard Vercel est protégé par compte Vercel (2FA disponible). Aucun accès SSH nécessaire (architecture PaaS — pas de serveur à gérer) mais la CLI de Vercel est activé sur le projet en local.

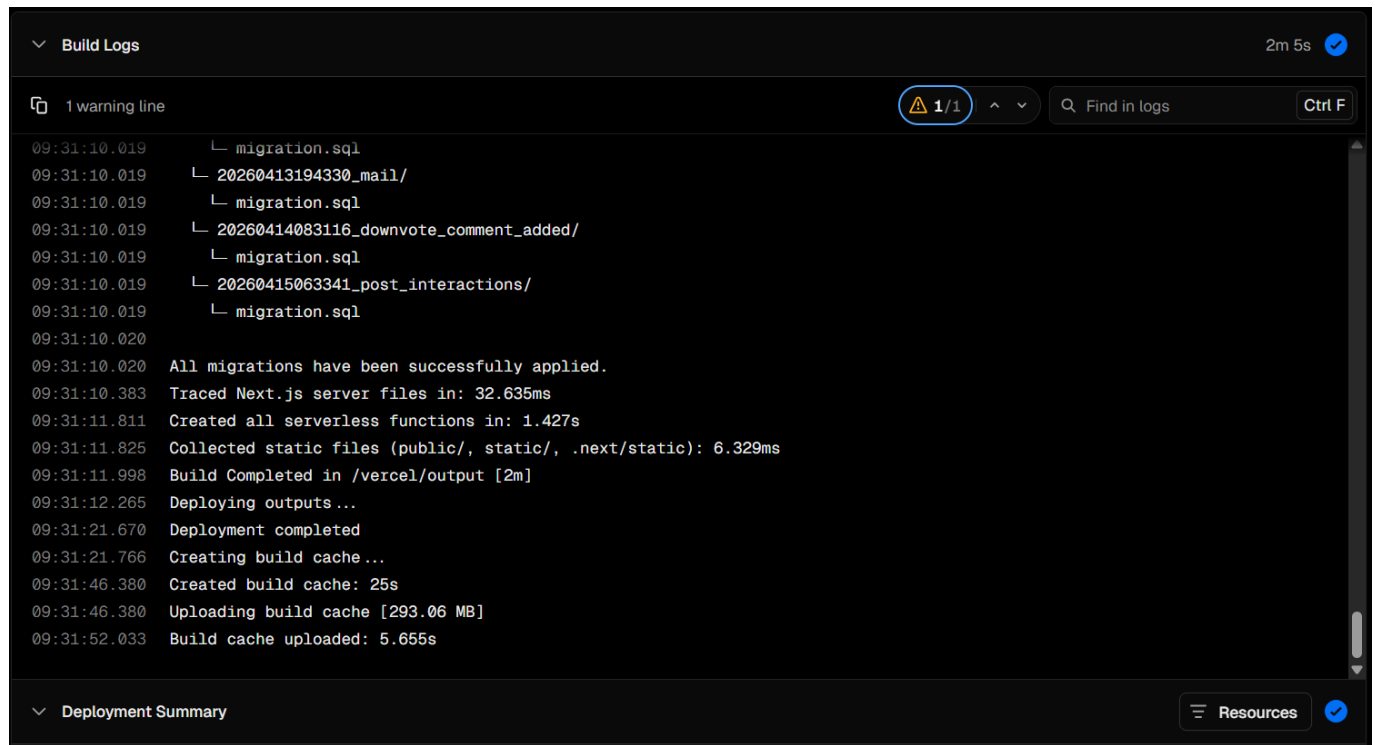
4. Exploitabilité / maintenance

Mise à jour depuis GitHub

Tout déploiement est déclenché automatiquement par un push GitHub sur la branche correspondante. Aucune intervention manuelle requise pour déployer.

Logs et supervision

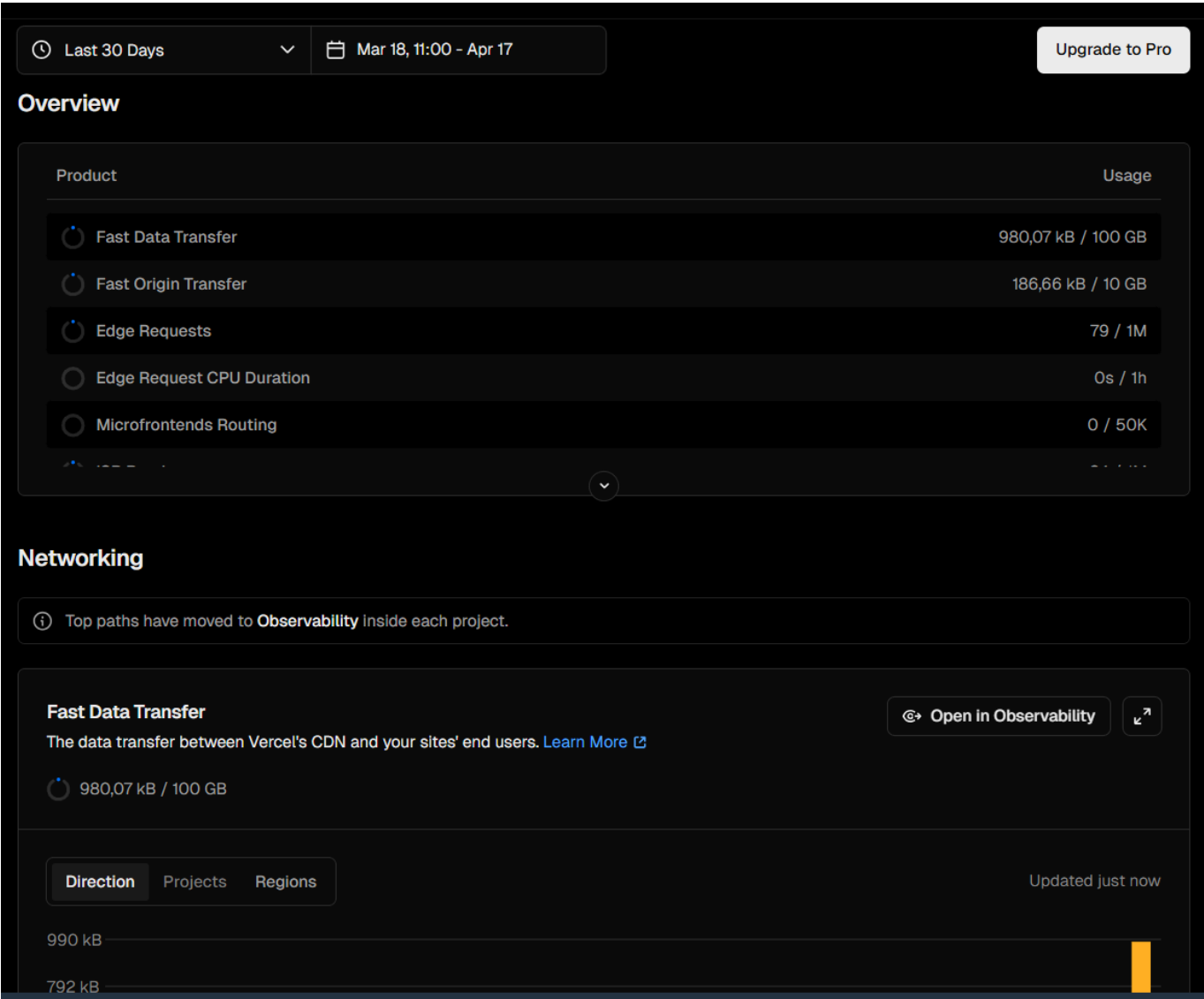
- **Build logs** : disponibles dans le dashboard Vercel pour chaque déploiement

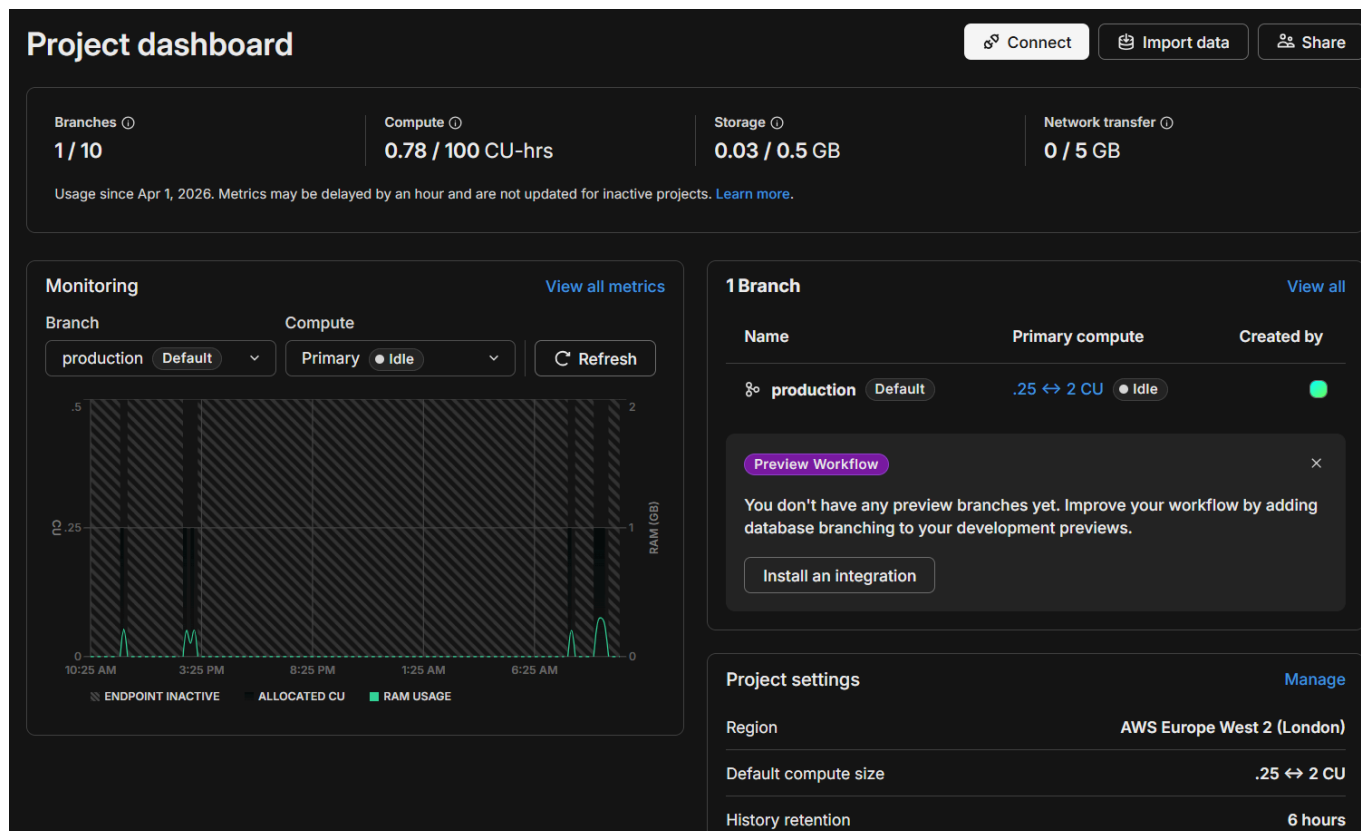


The screenshot displays the Vercel Build Logs interface. At the top, it shows 'Build Logs' with a dropdown arrow, a timer at '2m 5s', and a status icon. Below this, there's a section for '1 warning line' with a warning icon and '1/1' indicator. The main log area shows a series of timestamps and messages: '09:31:10.019' followed by several 'migration.sql' files being processed, '09:31:10.020' with the message 'All migrations have been successfully applied.', '09:31:10.383' with 'Traced Next.js server files in: 32.635ms', '09:31:11.811' with 'Created all serverless functions in: 1.427s', '09:31:11.825' with 'Collected static files (public/, static/, .next/static): 6.329ms', '09:31:11.998' with 'Build Completed in /vercel/output [2m]', '09:31:12.265' with 'Deploying outputs...', '09:31:21.670' with 'Deployment completed', '09:31:21.766' with 'Creating build cache...', '09:31:46.380' with 'Created build cache: 25s', '09:31:46.380' with 'Uploading build cache [293.06 MB]', and '09:31:52.033' with 'Build cache uploaded: 5.655s'. At the bottom, there's a 'Deployment Summary' section with a 'Resources' button and a status icon.

```
09:31:10.019    └─ migration.sql
09:31:10.019    └─ 20260413194330_mail/
09:31:10.019    └─ migration.sql
09:31:10.019    └─ 20260414083116_downvote_comment_added/
09:31:10.019    └─ migration.sql
09:31:10.019    └─ 20260415063341_post_interactions/
09:31:10.019    └─ migration.sql
09:31:10.020
09:31:10.020 All migrations have been successfully applied.
09:31:10.383 Traced Next.js server files in: 32.635ms
09:31:11.811 Created all serverless functions in: 1.427s
09:31:11.825 Collected static files (public/, static/, .next/static): 6.329ms
09:31:11.998 Build Completed in /vercel/output [2m]
09:31:12.265 Deploying outputs...
09:31:21.670 Deployment completed
09:31:21.766 Creating build cache...
09:31:46.380 Created build cache: 25s
09:31:46.380 Uploading build cache [293.06 MB]
09:31:52.033 Build cache uploaded: 5.655s
```

- **Runtime logs** : fonctions serverless loguées en temps réel sur Vercel (filtrage par niveau, source, status code)





Maintenance

La maintenance est g r e directement via une variable d'environnement stock e sur **Doppler** et v rifie si elle est   true pour afficher la page /maintenance sur lency.net en production ou sur la pr -production staging.lency.net

Proc dure d'acc s serveur

Architecture PaaS — pas de serveur   administrer directement. L'acc s se fait via :

1. Dashboard Vercel pour g rer l'h bergement, nom de domaine, SSL, Analytics etc.
2. Dashboard Neon.com pour la base de donn es
3. Dashboard Doppler pour les secrets d'environnements
4. CLI Vercel pour les op rations avanc es directement depuis l'IDE

5. Dossier de rendu

Solution retenue

La solution retenue repose sur une architecture **PaaS enti rement manag e**, sans serveur   administrer :

- **Vercel** comme plateforme d'h bergement et de d ploiement continu pour l'application **Next.js 15**
- **Neon.com** pour la base de donn es **PostgreSQL serverless**, avec deux instances isol es (production / staging)
- **Prisma ORM** pour la gestion du sch ma de donn es et les migrations version es
- **Doppler** pour la gestion centralis e des secrets d'environnement (3 projets : [lency](#), [lency-staging](#), [lency-development](#))
- **Resend** pour l'envoi d'emails transactionnels depuis support@mail.lency.net

- **Better Auth** pour l'authentification avec sessions sécurisées

Le déploiement est entièrement automatisé via un pipeline CI/CD GitHub géré par Vercel vers le déploiement hébergé sur Vercel : chaque push sur `main` ou `staging` déclenche un build, une migration Prisma et un déploiement sans intervention manuelle.

Étapes de déploiement

1. **Connexion GitHub → Vercel** : lier le dépôt privé `tim-vdb/Lency` aux deux projets Vercel (`lency` et `lency-staging`)
 2. **Configuration des branches** : `main` → `lency.net` (production), `staging` → `staging.lency.net`
 3. **Création des bases de données** sur Neon.com (instances prod et staging séparées), récupération des `DATABASE_URL`
 4. **Configuration Doppler** : création des trois projets avec toutes les variables d'environnement (DB, auth, API keys, SMTP...)
 5. **Intégration Doppler ↔ Vercel** : activation de la synchronisation automatique des secrets au moment du build via le dashboard Doppler
 6. **Configuration du domaine** `lency.net` sur Vercel, ajout des enregistrements DNS chez Vercel
 7. **Ajout des sous-domaines** : `staging.lency.net` et `mail.lency.net` (pour Resend)
 8. **Configuration email** : ajout des enregistrements SPF, DKIM et DMARC dans les DNS pour `mail.lency.net` via Resend
 9. **Vérification SSL** : certificats Let's Encrypt provisionnés automatiquement par Vercel sur tous les domaines
-

Difficultés rencontrées

- **Adapter Prisma pour Neon serverless** : Neon fonctionne via WebSocket en mode serverless, ce qui nécessite `@neondatabase/serverless` et `@prisma/adaptor-neon` à la place du driver PostgreSQL classique. La configuration de l'adaptateur dans le client Prisma a demandé des ajustements spécifiques.
 - **Migrations Prisma en production** : l'exécution des migrations au moment du build Vercel (et non au runtime) a imposé la création d'un script `vercel-build` dédié, pour garantir que le schéma DB est toujours synchronisé avec le code déployé.
 - **Injection des secrets via Doppler** : la configuration de la synchronisation Doppler → Vercel nécessite une intégration spécifique côté Doppler. Comprendre la priorité entre les variables définies directement dans Vercel et celles injectées par Doppler a nécessité des tests.
 - **Configuration DNS et emails** : la mise en place des enregistrements SPF, DKIM et DMARC pour `mail.lency.net` a requis une configuration précise côté Vercel et Resend pour éviter que les emails atterrissent en spam.
 - **Gestion des conflits de merge** : le workflow multi-branches (`main` / `staging` / `dev` / `features`) a occasionné des problématiques à réfléchir.
-

Justification des choix

Choix	Justification
Vercel (PaaS)	Compatibilité native avec Next.js (même éditeur), CDN global intégré, scaling automatique, SSL automatique, 0 gestion de serveur, intégration CI/CD GitHub en quelques clics géré via Vercel
Neon.com (PostgreSQL serverless)	Serverless adapté aux charges variables d'un SaaS, pricing à l'usage, instances isolées par environnement, compatibilité totale avec Prisma via adaptateur officiel
Prisma ORM	Type-safety complète avec TypeScript, migrations versionnées et traçables dans Git, Prisma Studio pour visualiser la DB en développement
Doppler	Centralisation des secrets multi-environnements en un seul outil, intégration native Vercel, aucun secret dans le code ni dans <code>.env</code> commité
Better Auth	Bibliothèque d'authentification moderne et flexible pour Next.js App Router, sessions sécurisées, extensible (OAuth, 2FA...)
Resend	Envoi d'emails transactionnels fiable via API REST simple, support SPF/DKIM/DMARC natif, taux de délivrabilité élevé